Криптографія

Комп’ютерний практикум №4

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Виконали:
студенти групи ФБ-41
Александрова М. та Проценко Д.

Мета: ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Хід роботи

1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту. В якості датчика випадкових чисел використовуйте вбудований генератор псевдовипадкових чисел вашої мови програмування. В якості тесту перевірки на простоту рекомендовано використовувати тест Міллера-Рабіна із попередніми пробними діленнями. Тести необхідно реалізовувати власноруч, використання готових реалізацій тестів не дозволяється.

Допоміжні арифметичні функції:
Швидке піднесення до степеня за модулем (a^b mod m) за схемою Горнера

```python
def modpow(a, b, m):
    """Швидке піднесення до степеня a^b mod m (схема Горнера)"""
    y = 1
    for bit in bin(b)[2:]:
        y = (y * y) % m
        if bit == '1':
            y = (y * a) % m
    return y
```

Показник степеня b подається у двійковому вигляді, і його біти опрацьовуються від старшого до молодшого. На кожному кроці проміжний результат y підноситься до квадрата за модулем m, а якщо поточний біт дорівнює 1, то додатково множиться на a за модулем m. Початкове значення y = 1.

Усі проміжні обчислення виконуються за модулем m, тому проміжні значення ніколи не перевищують m. Для k-бітного показника степеня потрібно не більше k піднесень до квадрата та не більше k множень, тобто складність становить O(log b) замість O(b) при послідовному множенні.

Функція є основою всіх обчислень у програмі: вона використовується в тесті Міллера-Рабіна (обчислення x^d mod p та послідовності квадратів), а також при шифруванні, розшифруванні, створенні й перевірці цифрового підпису та в протоколі розсилання ключів.

```python
def gcd(a, b):
    """Алгоритм Евкліда."""
    while b:
        a, b = b, a % b
    return a
```

Алгоритм Евкліда для знаходження найбільшого спільного дільника двох чисел (функція буде потрібна для перевірки взаємної простоти двох чисел: у тесті Міллера-Рабіна, RSA, перевірці ключа)

```python
def egcd(a, b):
    """Розширений алгоритм Евкліда: повертає (g, x, y), де a*x + b*y = g."""
    x0, x1, y0, y1 = 1, 0, 0, 1
    while b:
        q = a // b
        a, b = b, a - q * b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0
```

Розширений алгоритм Евкліда. Він також повертає найбільший спільний дільник, але, окрім цього, також додатково знаходить коефіцієнти, за якими обчислюється обернений елемент (секретний ключ RSA).

```python
def modinv(a, m):
    """Обернений елемент a^(-1) mod m."""
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError("Оберненого елемента не існує (gcd != 1)")
    return x % m
```

Функція для знаходження оберненого елемента за розширеним алгоритмом Евкліда.

<u>Тести перевірки на простоту:</u>

RSA працює лише якщо p і q прості. Тому після генерації випадкового числа треба переконатися, що воно дійсно просте. Для цього ми використовуємо такі перевірки:

- **тест пробних ділень**

```python
SMALL_PRIMES = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47,
                53, 59, 61, 67, 71, 73, 79, 83, 89, 97]


def trial_division(n):
    """
    Пробні ділення на малі прості.
    Повертає (True, None), якщо n пройшло перевірку,
    або (False, p), де p – малий дільник, що відкинув n.
    """
    for p in SMALL_PRIMES:
        if n == p:
            return True, None
        if n % p == 0:
            return False, p
    return True, None
```

Це попередній фільтр. Ми перевіряємо, чи ділиться n на малі прості числа (2, 3, 5, ..., 97). Якщо ділиться, число точно складене, і Міллер-Рабін для нього запускати не потрібно.
Рядок if n == p: return True, None потрібен для малих n. Саме просте число 7, наприклад, ділиться на 7, але воно не складене. Функція повертає пару: (True, None), якщо число пройшло фільтр, або (False, p) з дільником. Дільник ми зберігаємо, щоб у звіті вказати причину відкидання кандидата.
Однак, проходження пробних ділень не означає, що число просте, а лише що в нього немає малих дільників (тобто чисел з нашого списку SMALL_PRIMES).

- **перевірка, чи є p сильно псевдопростим за однією основою**

```python
def is_strong_pseudoprime(p, d, s, x, verbose=False):
    """
    Чи є p сильно псевдопростим за основою x (p - 1 = d * 2^s).
    """
    xr = modpow(x, d, p)
    if verbose:
        print(f"    x^d mod p = {xr}   (r = 0)")
    if xr == 1 or xr == p - 1:
        if verbose:
            print("    x^d mod p = ±1  =>  p сильно псевдопросте за цією основою")
        return True
    for r in range(1, s):
        xr = (xr * xr) % p
        if verbose:
            print(f"    x_{r} = x_{r-1}^2 mod p = {xr}")
        if xr == p - 1:
            if verbose:
                print(f"    x_{r} = -1  =>  p сильно псевдопросте за цією основою")
            return True
        if xr == 1:
            if verbose:
                print(f"    x_{r} = 1  =>  p НЕ сильно псевдопросте")
            return False
    if verbose:
        print("    -1 не знайдено  =>  p НЕ сильно псевдопросте")
    return False
```

За малою теоремою Ферма для простого p виконується x^(p−1) ≡ 1 (mod p)
Піднесення до степеня p − 1 можна розкласти на ланцюжок піднесень до квадрата (бо за простого модуля єдині квадратні корені з 1 це ±1, і тому x^(p−1) має «розпадатися» на ланцюжок x^d, x^(2d), ..., де кожен елемент є квадратом попереднього.):
p − 1 = d·2^s з непарним d.
Тоді x^(p−1) виходить з x^d послідовним піднесенням до квадрата s разів:
x^d → x^(2d) → x^(4d) → ... → x^(2^s·d) = x^(p−1) (тобто x^(p−1) «розпадається» на ланцюжок x^d, x^(2d), ..., де кожен елемент є квадратом попереднього).

За малою теоремою Ферма останній елемент дорівнює 1 (x^(p−1) ≡ 1 (mod p))
Для простого p в цьому ланцюжку є лише два дозволені варіанти:

1) або x^d ≡ 1 вже на початку;
2) або десь у ланцюжку з'являється −1 і далі після піднесення до квадрата виходить 1.

В коді:
xr = modpow(x, d, p) це x^d mod p. Якщо результат 1 або p − 1, то число сильно псевдопросте за цією основою.
Цикл for r in range(1, s) послідовно підносить до квадрата. Якщо отримали p − 1, то число проходить. Якщо отримали 1 (а попереднє значення було не ±1), число складене.
Якщо цикл закінчився і −1 так і не з'явилось, число складене.

(Останній член x^(p−1) окремо рахувати не потрібно. Якщо ми дійшли до нього, не побачивши −1, то або він не дорівнює 1 (порушено Ферма), або він 1, але отриманий з не ±1 (нетривіальний корінь). В обох випадках число складене.)

- **повний тест Міллера-Рабіна**

```python
def miller_rabin(p, k=20, verbose=False):
    """
    Імовірнісний тест Міллера–Рабіна з k випадковими основами.
    """
    if p < 2:
        return False, None
    if p in (2, 3):
        return True, None
    if p % 2 == 0:
        return False, 2
```

Що функція приймає p (число, яке перевіряємо), k (кількість випадкових основ (раундів), за замовчуванням 20), verbose вмикає друк проміжних значень (для звіту).
Повертає пару:
(True, None): p вважаємо простим;
(False, x): p складене, а x є свідком складеності (основа, яка це виявила). Для парного p свідком повертається 2 (дільник).

Граничні випадки:
- числа менші за 2 не прості
- 2 і 3 прості
- парні числа більші за 2 складені. Далі алгоритм розраховує на непарне p (щоб p − 1 було парним і мало розклад із s ≥ 1).

```python
    # Крок 0: p − 1 = d * 2^s, d непарне
    d, s = p - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
```

крок 0, розклад p − 1 = d·2^s

Ділимо d на 2, поки воно парне, і рахуємо кількість поділень. Це робиться один раз для всього p, а не для кожної основи, бо розклад від x не залежить.

```
if verbose:
    print(f"  p = {p}")
    print(f"  p - 1 = d * 2^s,  s = {s},  d = {d}")
```

Просто виводить розклад.

Цикл по основах:

```
for i in range(k):
    # Крок 1: x з інтервалу 1 < x < p
    x = random.randint(2, p - 1)
```

Вибираємо випадкову основу з інтервалу 1 < x < p. Метод randint включає обидва кінці, тож x може бути від 2 до p − 1. Основа кожного раунду обирається незалежно від попередніх.

```
    g = gcd(x, p)
    show = verbose and i == 0
    if show:
        print(f"  Основа x = {x},  gcd(x, p) = {g}")
    if g > 1:
        return False, x
```

Перевіряємо gcd(x, p). Якщо він більший за 1, то x і p мають спільний дільник, тож p складене, і ми одразу повертаємо False. Для простого p цього ніколи не станеться, бо всі x від 2 до p − 1 з ним взаємно прості. Тому ця перевірка лише швидше ловить деякі складені числа. Також вона гарантує умову gcd(x, p) = 1, потрібну для малої теореми Ферма, на якій тримається вся логіка.
Змінна show дорівнює True тільки для першої ітерації (i == 0) і лише якщо увімкнено verbose. Це забезпечує друк проміжних результатів тільки для першої основи, як вимагає завдання.

```
        # Крок 2: чи сильно псевдопросте за основою x
        if not is_strong_pseudoprime(p, d, s, x, verbose=show):
            return False, x
    return True, None
```

Перевіряємо, чи p сильно псевдопросте за основою x (ланцюжок x^d, x^(2d), ...). Якщо ні, то p точно складене, і повертаємо False та x як свідка. Одного провалу достатньо, решту основ не перевіряємо.
return True, None стоїть після циклу і виконується, лише якщо всі k основ пройшли. Тоді p вважаємо простим.

Функція шукає просте число заданої довжини в бітах:

```
def gen_prime(bits, k=20, rejected=None):
    """
    Випадкове просте число довжиною рівно `bits` біт.
    rejected – список, куди записуються кандидати, що не пройшли тести
    """
```

бере випадкове непарне число потрібної довжини і перевіряє числа підряд (x, x+2, x+4, ...), доки не знайде просте.

Параметри: bits це довжина числа, k це кількість основ Міллера-Рабіна, rejected це список, куди складаємо відкинутих кандидатів (для звіту).

```
    if rejected is None:
        rejected = []
```

Якщо список не передали, створюємо порожній.

```
    while True:
        x = random.getrandbits(bits)
        x |= (1 << (bits - 1))      # старший біт = 1  -> рівно `bits` біт
        x |= 1                      # непарне
```

Зовнішній цикл повторюється, поки не знайдемо просте. Три рядки формують кандидата:
getrandbits(bits) дає випадкове число до bits біт,
x |= (1 << (bits - 1)) ставить старший біт в 1. Без цього число могло б мати менше за 256 біт (наприклад, 250), а нам треба рівно 256. Операція |= це побітове АБО,
x |= 1 ставить молодший біт в 1, тобто число стає непарним. Парні числа простими не бувають, тож їх не перевіряємо.

```
        while x.bit_length() == bits:
```

Внутрішній цикл іде по кандидатах x, x+2, x+4, ..., доки число має рівно bits біт. Якщо додавання 2 призвело б до переповнення на 257-й біт (дуже рідкісний випадок, якщо випадкове x було біля 2^{256}), цикл завершується, і зовнішній while True бере нове випадкове x.

```
            ok, div = trial_division(x)
            if not ok:
                rejected.append((x, f"пробні ділення: ділиться на {div}"))
```

Пробні ділення. Якщо x ділиться на малий простий, записуємо в rejected з причиною і переходимо до наступного кандидата. Дорогий Міллер-Рабін для нього не запускаємо.

```
            else:
                ok, witness = miller_rabin(x, k)
                if ok:
                    return x
                rejected.append((x, f"Міллер–Рабін: основа {witness}"))
```

Міллер-Рабін. Якщо пробні ділення пройдено, запускаємо Міллера-Рабіна з k основами. Якщо він каже "просте", повертаємо x і працю закінчено. Якщо ні, записуємо кандидата зі свідком складеності.

```
            x += 2                      # наступне непарне число
```

Переходимо до наступного непарного числа.

Доповнення до п. 1. Для одного зі згенерованих простих чисел навести проміжні результати перевірки тестом Міллера-Рабіна для однієї з використаних основ.

Проміжні результати перевірки числа p_1 тестом Міллера-Рабіна (для однієї з основ)

```
=== П.1. Проміжні результати тесту Міллера–Рабіна для числа p1 ===
  p = 67381378254839111310633187717012888273658005628402671042514090789505459168857
  p − 1 = d * 2^s,  s = 3,  d =
842267228185488891382914846462661103420725070355033388031426134868818239610 7
  Основа x = 112153652401829426033202175609067460841947930323515518492349593683177807936 37,
gcd(x, p) = 1
    x^d mod p = 549534130857691471748177358232291062764026052498456994227336609203410957123 50
(r = 0)
    x_1 = x_0^2 mod p =
202270560626930745252832920925260396722893446773359130000263448028253424849 81
    x_2 = x_1^2 mod p =
673813782548391113106331877170128882736580056284026710425140907895054591688 56
    x_2 = −1  =>  p сильно псевдопросте за цією основою
Відкинуті кандидати (кількість): {'p': 59, 'q': 52, 'p1': 27, 'q1': 52} -> файл
rejected_candidates.txt
```

Параметр	**Значення**
p1	67381378254839111310633187717012888273658005628402671042514090789505459168857
$p_1 - 1 = d \cdot 2^s$	s = 3, d = 84226722818548889138291484646266110342072507035503338803142613486881823961 07
основа x	11215365240182942603320217560906746084194793032351551849234959368317780793637
gcd(x, p_1)	1
r = 0: $x^d \bmod p_1$	54953413085769147174817735823229106276402605249845699422733660920341095712350
r = 1: $x_1 = x_0^2 \bmod p_1$	20227056062693074525283292092526039672289344677335913000026344802825342484981
r = 2: $x_2 = x_1^2 \bmod p_1$	67381378254839111310633187717012888273658005628402671042514090789505459168856
Висновок	$x_2 \equiv -1 \pmod{p_1}$, тому p_1 сильно псевдопросте за основою x

Значення x_2 дорівнює $p_1 - 1$, тобто -1 за модулем p_1, тому число проходить тест за цією основою; далі ланцюжок не продовжується

2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини щонайменше 256 біт. При цьому пари чисел беруться так, щоб $pq \le p_1q_1$; p і q - прості числа для побудови ключів абонента A, p_1 і q_1 - абонента B.

Функція для генерації двох пар простих чисел p, q, p_1, q_1 довжини ≥256 біт так, щоб $pq \le p_1q_1$ (викликає функцію gen_prime):
(У протоколі розсилання ключів абонент A підписує значення k своїм секретним ключем (за модулем n), а потім шифрує результат відкритим ключем B (за модулем n_1). Щоб значення S, яке менше за n, гарантовано вмістилося в модуль n_1 і не зіпсувалося, потрібно $n_1 \ge n$, тобто $p_1q_1 \ge pq$. Тому A отримує меншу пару, а B більшу.)

```python
def generate_primes_for_AB(bits=256, k=20):
    """
    Повертає (p, q, p1, q1, rejected) з умовою p*q <= p1*q1.
    (p, q) — для абонента A, (p1, q1) — для абонента B.
    """
    rejected = {"p": [], "q": [], "p1": [], "q1": []}

    def distinct_pair(name1, name2):
        a = gen_prime(bits, k, rejected[name1])
        b = gen_prime(bits, k, rejected[name2])
        while b == a:
            b = gen_prime(bits, k, rejected[name2])
        return a, b

    p, q = distinct_pair("p", "q")
    p1, q1 = distinct_pair("p1", "q1")

    if p * q > p1 * q1:             # міняємо пари місцями
        p, q, p1, q1 = p1, q1, p, q
        rejected["p"], rejected["p1"] = rejected["p1"], rejected["p"]
        rejected["q"], rejected["q1"] = rejected["q1"], rejected["q"]
    return p, q, p1, q1, rejected
```

Функція повертає кортеж (p, q, p1, q1, rejected):
(p, q) для абонента A, (p_1, q_1) для абонента B;
rejected це словник із чотирма списками відкинутих кандидатів, по одному для кожного числа (для звіту).

Згенеровані прості числа:

```
=== П.2. Генерація простих чисел ===
p  = 71444528368637790105449912299511327176198295515194297080590552313424290115601
q  = 58034388710460041470371062817225876330420042497927397641792301798888754965763
p1 = 67381378254839111310633187717012888273658005628402671042514090789505459168857
q1 = 10515118411232117305228052120875318186373192065502776687859173453943685337245 1
p*q <= p1*q1: True
```

p = 71444528368637790105449912299511327176198295515194297080590552313424290115601
q = 58034388710460041470371062817225876330420042497927397641792301798888754965763
p1 = 67381378254839111310633187717012888273658005628402671042514090789505459168857
q1 = 10515118411232117305228052120875318186373192065502776687859173453943685337245 1

Умова p*q <= p1*q1 виконується.

Кандидати, що не пройшли тест, записані у файлі rejected_candidates.txt:
для кожного з чотирьох чисел (p, q, p1, q1) окремий блок із заголовком «Кандидати для p: відкинуто N», а далі рядки виду кандидат <- причина
Причина або «пробні ділення: ділиться на 3», або «Міллер-Рабін: основа ...» (це свідок складеності). У консолі виводиться лише кількість (59, 52, 27, 52).

```
=== Кандидати для p: відкинуто 59 ===
71444528368637790105449912299511327176198295515194297080590552313424290115483  <- пробні ділення: ділиться на 3
71444528368637790105449912299511327176198295515194297080590552313424290115485  <- пробні ділення: ділиться на 5
71444528368637790105449912299511327176198295515194297080590552313424290115487  <- пробні ділення: ділиться на 13
71444528368637790105449912299511327176198295515194297080590552313424290115489  <- пробні ділення: ділиться на 3
71444528368637790105449912299511327176198295515194297080590552313424290115491  <- Міллер-Рабін: основа
21139857373288060809838143944894958378948093128501369796153345091702245730183
71444528368637790105449912299511327176198295515194297080590552313424290115493  <- Міллер-Рабін: основа
43988517180466426390611039461028021164534732558448964986701032214959404035 60
71444528368637790105449912299511327176198295515194297080590552313424290115495  <- пробні ділення: ділиться на 3
71444528368637790105449912299511327176198295515194297080590552313424290115497  <- пробні ділення: ділиться на 7
71444528368637790105449912299511327176198295515194297080590552313424290115499  <- Міллер-Рабін: основа
24363449885362813107521001871421369699888480897899926733657225036554696502131
71444528368637790105449912299511327176198295515194297080590552313424290115501  <- пробні ділення: ділиться на 3
71444528368637790105449912299511327176198295515194297080590552313424290115503  <- пробні ділення: ділиться на 83
71444528368637790105449912299511327176198295515194297080590552313424290115505  <- пробні ділення: ділиться на 5
71444528368637790105449912299511327176198295515194297080590552313424290115507  <- пробні ділення: ділиться на 3
71444528368637790105449912299511327176198295515194297080590552313424290115509  <- Міллер-Рабін: основа
48510250072751161947068170395423630323075760091564459859461034008944873848971
```

Наприклад, кандидата для p
71444528368637790105449912299511327176198295515194297080590552313424290115483 було відкинуто на етапі пробних ділень (було виявлено, що воно ділиться на 3, одне з малих простих чисел зі списку SMALL_PRIMES)

Кандидат для q
58034388710460041470371062817225876330420042497927397641792301798888754965659 пройшов пробні ділення (тобто малих дільників до 97 у нього немає), але тест Міллера-Рабіна його відкинув.
Основа 875318278678431093136062822825001698025462598412514372437985 1958261375145479 це та випадкова основа x, на якій тест виявив, що кандидат складений. Таку основу називають свідком складеності.

3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e). За допомогою цієї функції побудувати схеми RSA для абонентів A і B - тобто, створити та зберегти для подальшого використання відкриті ключі (e, n), (e_1, n_1) та секретні d і d_1. Для однієї зі згенерованих пар ключів навести значення основних параметрів та перевірити виконання співвідношень, необхідних для коректної роботи RSA.

```python
def GenerateKeyPair(p, q, e=None):
    """
    Повертає (public, private):
        public  = (n, e)
        private = (d, p, q)
    """
```

Ця функція за двома простими числами будує ключі RSA

```
    n = p * q
    phi = (p - 1) * (q - 1)
```

n це модуль, який входить у відкритий ключ. φ(n) це функція Ойлера, яка для добутку двох простих дорівнює (p − 1)(q − 1). Вона потрібна лише для обчислення d і є секретною: знаючи φ(n), можна знайти d, тому її не публікують.

```
    if e is None:
        e = 2 ** 16 + 1
```

За замовчуванням беремо e = 65537 = 2^{16} + 1. В сучасних системах майже завжди обирають саме його: воно просте, має лише два одиничні біти (тому шифрування за схемою Горнера майже не коштує), і при цьому достатньо велике.

```
    while not (2 <= e <= phi - 1 and gcd(e, phi) == 1):
        e = random.randint(2, phi - 1)
```

Перевіряємо умови 2 ≤ e ≤ φ(n) − 1 і gcd(e, φ(n)) = 1. Для нашого 65537 вони майже завжди виконуються одразу, і цикл не запускається (перша умова істинна, друга теж, not дає False). Якщо ж gcd ≠ 1 (рідкісний випадок, коли φ(n) ділиться на 65537), то підбираємо випадкове e, поки не підійде. Так функція працює й тоді, коли користувач передав власне e, яке не підходить.

```
    d = modinv(e, phi)
    return (n, e), (d, p, q)
```

Секретний ключ d це обернений до e за модулем φ(n): e·d ≡ 1 (mod φ(n)). Він обчислюється розширеним алгоритмом Евкліда. Повертаємо дві частини:

- відкритий ключ (n, e): його знає кожен;
- секретний ключ (d, p, q): таємний. Для розшифрування потрібне лише d (і n), але p і q зберігаємо, бо цього вимагає завдання («секретний ключ (d, p, q)»).

```
def check_key(public, private, name="", verbose=True):
    """Перевірка співвідношень, потрібних для коректної роботи RSA."""
    n, e = public
    d, p, q = private
    phi = (p - 1) * (q - 1)
    M = random.randint(2, n - 2)
    C = modpow(M, e, n)
    checks = {
        "p * q == n": p * q == n,
        "gcd(e, phi(n)) == 1": gcd(e, phi) == 1,
        "e * d ≡ 1 (mod phi(n))": (e * d) % phi == 1,
        "(M^e)^d ≡ M (mod n)": modpow(C, d, n) == M,
    }
    if verbose:
        print(f"--- Параметри ключа {name} ---")
        print(f"p      = {p}")
        print(f"q      = {q}")
        print(f"n      = {n}   ({n.bit_length()} біт)")
        print(f"phi(n) = {phi}")
        print(f"e      = {e}")
        print(f"d      = {d}")
        print(f"Тест: M = {M}\n      C = M^e mod n = {C}\n      C^d mod n = {modpow(C, d, n)}")
        for text, res in checks.items():
            print(f"  {text}: {'OK' if res else 'ПОМИЛКА'}")
    return all(checks.values())
```

Ця функція перевіряє, що ключ справді працює.
Спершу дістає параметри й перераховує φ(n). Потім бере випадкове тестове повідомлення M і шифрує його: C = M^e mod n.
Далі в словнику checks чотири умови:

Перевірка	**Що доводить**
p * q == n	модуль складено правильно
gcd(e, phi) == 1	e має обернений елемент за модулем φ(n)
e * d ≡ 1 (mod phi)	d справді обернений до e
(M^e)^d ≡ M (mod n)	головна перевірка: шифрування з наступним розшифруванням повертає вихідне повідомлення

```
def save_keys(filename, public, private):
    n, e = public
    d, p, q = private
    with open(filename, "w") as f:
        json.dump({"n": hex(n), "e": hex(e), "d": hex(d),
                   "p": hex(p), "q": hex(q)}, f, indent=2)
```

Ця функція зберігає ключі у файл JSON. Числа записуються в шістнадцятковому вигляді (hex)

```
def load_keys(filename):
    with open(filename) as f:
        j = json.load(f)
    n, e, d, p, q = (int(j[k], 16) for k in ("n", "e", "d", "p", "q"))
    return (n, e), (d, p, q)
```

Ця функція робить зворотне: читає файл і перетворює рядки назад у числа через int(..., 16). Повертає ту саму пару (n, e), (d, p, q), що й GenerateKeyPair.

```
=== П.3. Ключові пари RSA ===
--- Параметри ключа A ---
p      = 7144452836863779010544991229951132717619829551519429708059055231342429011560 1
q      = 5803438871046004147037106281722587633042004249792739764179230179888875496576 3
n      =
4146239530581015130055747338654753717802798314001821161777451943081077533331880151642990006772654302217053545662699751428897216465843965557581546067168563   (511 біт)
phi(n) =
4146239530581015130055747338654753717802798314001821161777451943081077533331750672725910908941078481241936808459193133090884094771121582703469233022087200
e      = 65537
d      =
1942254811615380082895333068292734472687884832077390018868238928431101214173440127755092007636771737707363169197510248956927258029409838549163151407267 3
Тест: M =
2828618279345309782225755415789359222618916248936162976514521594950963524778829757256058367166562413663721261349828444128660962772544912910010165216878615
      C = M^e mod n =
1124545880117802286301678872728839447997490935525976011351682950721631325807568656078333072249058956702590691436642502187483628214113135129792671484488919
      C^d mod n =
2828618279345309782225755415789359222618916248936162976514521594950963524778829757256058367166562413663721261349828444128660962772544912910010165216878615
  p * q == n: OK
  gcd(e, phi(n)) == 1: OK
  e * d ≡ 1 (mod phi(n)): OK
  (M^e)^d ≡ M (mod n): OK
```

Параметри ключа A:

p	7144452836863779010544991229951132717619829551519429708059055 2313424290115601
q	5803438871046004147037106281722587633042004249792739764179230 1798888754965763
n	4146239530581015130055747338654753717802798314001821161777451 9430810775333318801516429900067726543022170535456626997514288 9721646584396555758154606716856 3 (511 біт)
φ(n)	4146239530581015130055747338654753717802798314001821161777451 9430810775333317506727259109089410784812419368084591931330908 8409477112158270346923302208720 0
e	65537
d	1942254811615380082895333068292734472687884832077390018868238 9284311012141734401277550920076367717377073631691975102489569 2725802940983854916315140726 73

4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів A і B. Кожна з операцій (шифрування, розшифрування, створення цифрового підпису, перевірка цифрового підпису) повинна бути реалізована окремою процедурою, на вхід до якої повинні подаватись лише ті ключові дані, які необхідні для її виконання.
За допомогою датчика випадкових чисел вибрати відкрите повідомлення M і знайти криптограму для абонентів A і B, перевірити правильність розшифрування. Скласти для A і B повідомлення з цифровим підписом і перевірити його.

```python
# ПУНКТ 4. Базові криптографічні операції RSA
def Encrypt(M, public_key):
    """Шифрування повідомлення M відкритим ключем (e, n)."""
    e, n = public_key
    if not (0 <= M < n):
        raise ValueError("Повідомлення M повинно бути в межах 0 <= M < n")
    return modpow(M, e, n)

def Decrypt(C, private_key):
    """Розшифрування криптограми C секретним ключем (d, n)."""
    d, n = private_key
    if not (0 <= C < n):
        raise ValueError("Криптограма C повинна бути в межах 0 <= C < n")
    return modpow(C, d, n)

def Sign(M, private_key):
    """Створення повідомлення з цифровим підписом (M, S) секретним ключем (d, n)."""
    d, n = private_key
    if not (0 <= M < n):
        raise ValueError("Повідомлення M повинно бути в межах 0 <= M < n")
    S = modpow(M, d, n)
    return (M, S)

def Verify(M, S, public_key):
    """Перевірка цифрового підпису S відкритим ключем (e, n)."""
    e, n = public_key
    return modpow(S, e, n) == M
```

Для виконання завдання попередньо збережені ключі абонентів A та B зчитуються з JSON-файлів за допомогою функції load_keys, після чого реалізовано чотири окремі криптографічні процедури: rsa_encrypt (шифрування), rsa_decrypt (розшифрування), rsa_sign (формування повідомлення з цифровим підписом) та rsa_verify (перевірка підпису). Відповідно до умови, на вхід кожної функції передаються виключно необхідні для обчислень ключові параметри — (e, n) або (d, n), а також виконується перевірка належності повідомлення чи криптограми допустимому діапазону $[0, n - 1]$

```
87      # --- Перевірка П.4 ---
88      print("=== П.4. Шифрування та цифровий підпис RSA ===")
89      M = random.randint(2, n - 1)
90      print(f"Випадкове відкрите повідомлення M = {M}\n")
91
92      C_A = rsa_encrypt(M, e, n)
93      M_dec_A = rsa_decrypt(C_A, d, n)
94      _, S_A = rsa_sign(M, d, n)
95      valid_A = rsa_verify(M, S_A, e, n)
96
97      print("--- Абонент A ---")
98      print(f"Криптограма C_A      = {C_A}")
99      print(f"Розшифроване M_dec_A = {M_dec_A}")
100     print(f"Розшифрування вірне? = {'OK' if M_dec_A == M else 'ПОМИЛКА'}")
101     print(f"Цифровий підпис S_A  = {S_A}")
102     print(f"Підпис дійсний?      = {'OK' if valid_A else 'ПОМИЛКА'}\n")
103
104     C_B = rsa_encrypt(M, e1, n1)
105     M_dec_B = rsa_decrypt(C_B, d1, n1)
106     _, S_B = rsa_sign(M, d1, n1)
107     valid_B = rsa_verify(M, S_B, e1, n1)
108
109     print("--- Абонент B ---")
110     print(f"Криптограма C_B      = {C_B}")
111     print(f"Розшифроване M_dec_B = {M_dec_B}")
112     print(f"Розшифрування вірне? = {'OK' if M_dec_B == M else 'ПОМИЛКА'}")
113     print(f"Цифровий підпис S_B  = {S_B}")
114     print(f"Підпис дійсний?      = {'OK' if valid_B else 'ПОМИЛКА'}\n")
115
```

На наведеному фрагменті коду реалізовано тестування роботи криптографічних операцій для обох учасників взаємодії. За допомогою генератора випадкових чисел формується відкрите повідомлення $M \in [2, n-1]$, яке почергово шифрується, розшифровується, підписується та перевіряється спочатку на ключах абонента A — (e, n) та (d, n), а потім на ключах абонента B — (e_1, n_1) та (d_1, n_1). Для кожного абонента програма автоматично звіряє розшифроване значення з початковим повідомленням M та перевіряє дійсність цифрового підпису, виводячи відповідні результати в консоль.

5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA. Протоколи роботи кожного учасника (відправника та приймаючого) повинні бути реалізовані у вигляді окремих процедур, на вхід до яких повинні подаватись лише ті ключові дані, які необхідні для виконання.
Перевірити роботу програм для випадково обраного ключа 0 < k < n.
У звіті для одного виконання протоколу показати послідовність переданих та обчислених значень і зазначити, використанням якого ключа забезпечується конфіденційність та підтвердження справжності відправника.

```python
# ПУНКТ 5. Протокол конфіденційного розсилання ключів з підтвердженням справжності
def SendKey(k, sender_private_key, receiver_public_key):
    """
    Процедура відправника (абонент A):
    1. Підписує ключ k своїм секретним ключем (d, n) -> S.
    2. Шифрує k та S відкритим ключем отримувача B (e1, n1) -> (k1, S1).
    """
    _, n = sender_private_key
    if not (0 < k < n):
        raise ValueError("Ключ k повинен задовольняти умову 0 < k < n")

    _, S = Sign(k, sender_private_key)
    k1 = Encrypt(k, receiver_public_key)
    S1 = Encrypt(S, receiver_public_key)
    return S, (k1, S1)


def ReceiveKey(packet, receiver_private_key, sender_public_key):
    """
    Процедура приймаючого (абонент B):
    1. Розшифровує (k1, S1) своїм секретним ключем (d1, n1) -> (k', S').
    2. Перевіряє підпис S' відкритим ключем відправника A (e, n).
    """
    k1, S1 = packet
    k_received = Decrypt(k1, receiver_private_key)
    S_received = Decrypt(S1, receiver_private_key)
    is_authentic = Verify(k_received, S_received, sender_public_key)
    return k_received, S_received, is_authentic
```

```python
    # --- Перевірка П.5 ---
    print("=== П.5. Протокол розсилання ключів з підтвердженням справжності (A -> B) ===")
    k = random.randint(1, n - 1)

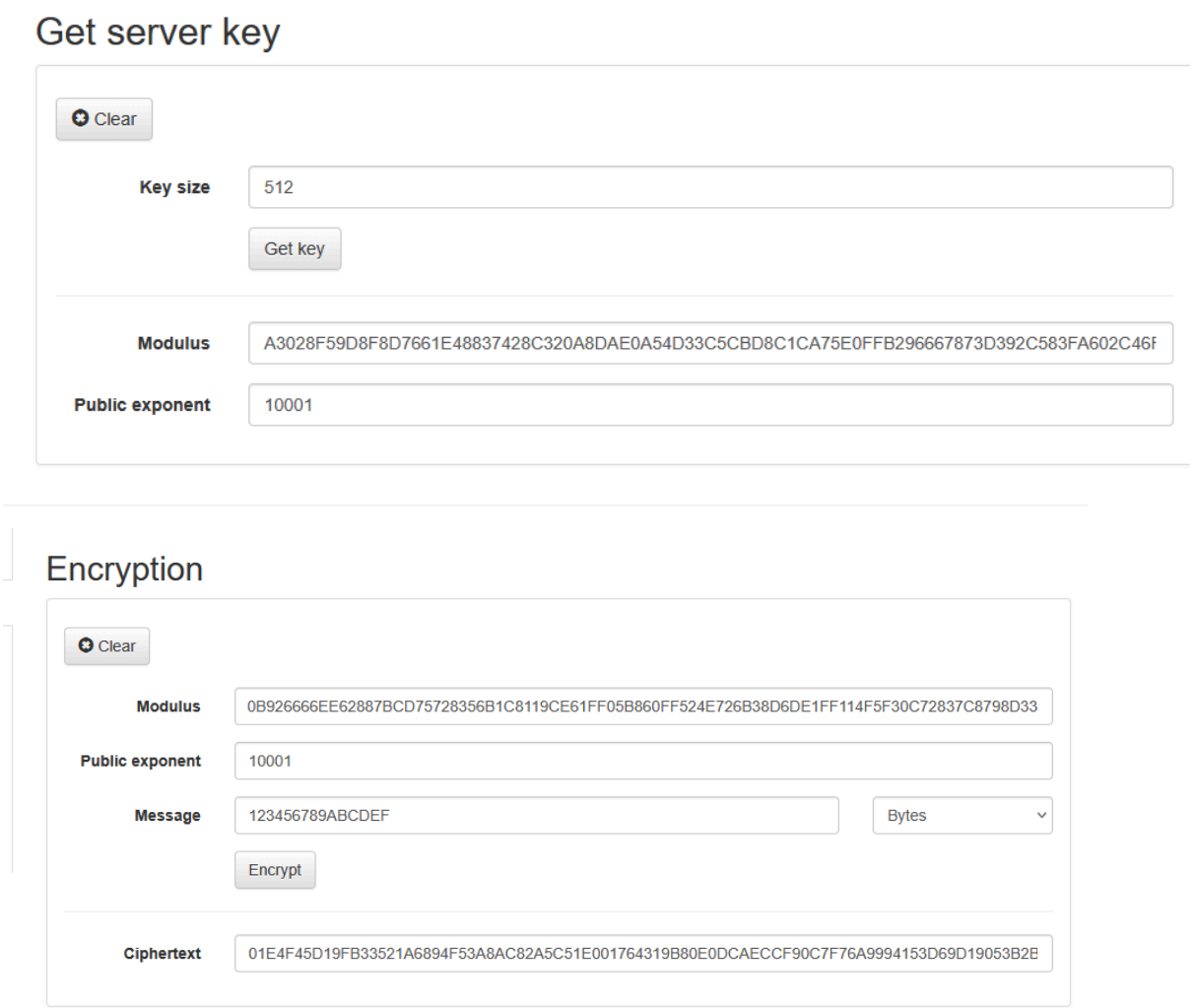
    S, packet = SendKey(k, priv_A, pub_B)
    k1, S1 = packet
    k_recv, S_recv, is_authentic = ReceiveKey(packet, priv_B, pub_A)

    print(f"1. Абонент A вибрав випадковий ключ k   = {k}")
    print(f"2. Абонент A обчислив підпис S          = {S}")
    print(f"3. Передано у відкритий канал k1        = {k1}")
    print(f"   Передано у відкритий канал S1        = {S1}")
    print(f"4. Абонент B розшифрував ключ k'        = {k_recv}")
    print(f"   Абонент B розшифрував підпис S'      = {S_recv}")
    print(f"5. Підпис відправника A підтверджено?   = {'ОК' if is_authentic else 'ПОМИЛКА'}")
    print(f"   Отриманий ключ k' збігається з k?    = {'ОК' if k == k_recv else 'ПОМИЛКА'}")
```

у функції SendKey спочатку перевіряється умова 0 < k < n (із генерацією ValueError при виході за межі), далі викликом Sign обчислюється підпис S, а значення k та S шифруються функцією Encrypt на відкритому ключі отримувача з поверненням пакета (k1, S1). У функції ReceiveKey вхідний кортеж packet розпаковується на k1 і S1, які послідовно розшифровуються викликами Decrypt на секретному ключі отримувача у змінні k_received та S_received, після чого функція Verify повертає прапорець дійсності підпису is_authentic. У тестовому блоці (Перевірка П.5) через random.randint(1, n - 1) генерується випадковий ключ k, виконується обмін викликами SendKey(k, priv_A, pub_B) та ReceiveKey(packet, priv_B, pub_A), а виведення в консоль через умовні вирази перевіряє істинність is_authentic та рівність k == k_recv

```
=== П.5. Протокол розсилання ключів з підтвердженням справжності (A -> B) ===
1. Абонент A вибрав випадковий ключ k   = 962877778047918088695228557236733745904764149077982260981088934771695421041445426733639185632113305243859491104841891194456033781390203237054575461328189
2. Абонент A обчислив підпис S          = 928983343976803763420935515533591014696387456897617087303694859467255889118171625674031851314725422207294789470198720437514752999768999739990833627730868
3. Передано у відкритий канал k1        = 27524287759918942098781382105733520338238204427261545247400708512883073629027517658718254179360585535686234156568645119324394426798755590228113683470764914
   Передано у відкритий канал S1        = 4049371594838687665314760350695503784318923602171262055464466421093377766806907580763224632190731323928650938064950898654064272061502568635255888540993359
4. Абонент B розшифрував ключ k'        = 962877778047918088695228557236733745904764149077982260981088934771695421041445426733639185632113305243859491104841891194456033781390203237054575461328189
   Абонент B розшифрував підпис S'      = 928983343976803763420935515533591014696387456897617087303694859467255889118171625674031851314725422207294789470198720437514752999768999739990833627730868
5. Підпис відправника A підтверджено?   = OK
   Отриманий ключ k' збігається з k?    = OK
```

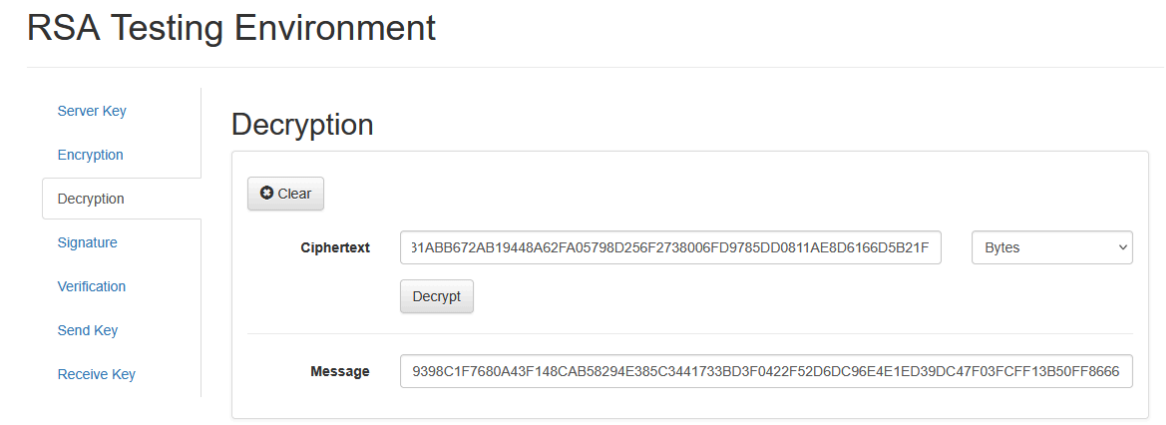
Перейдемо до перевірок на сайті http://asymcryptwebservice.appspot.com/?section=rsa



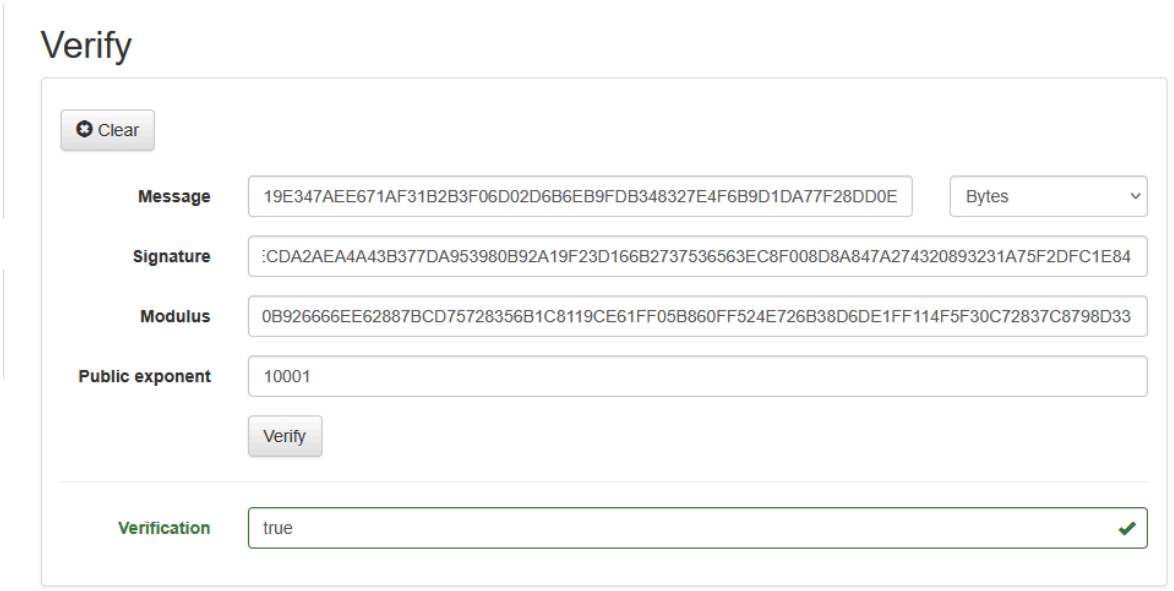
```
--- Encryption ---
Modulus:         4F2A658BA417B28938E5CE40620F8CA0720C9DEE290B926666EE62887BCD75728356B1C8119CE61FF05B860FF524E726B38D6DE1FF114F5F30C72837C8798D33
Public exponent: 10001
Розшифроване локально M: 123456789ABCDEF
```

Шифруємо за нашим ключем

Успішно розшифровує

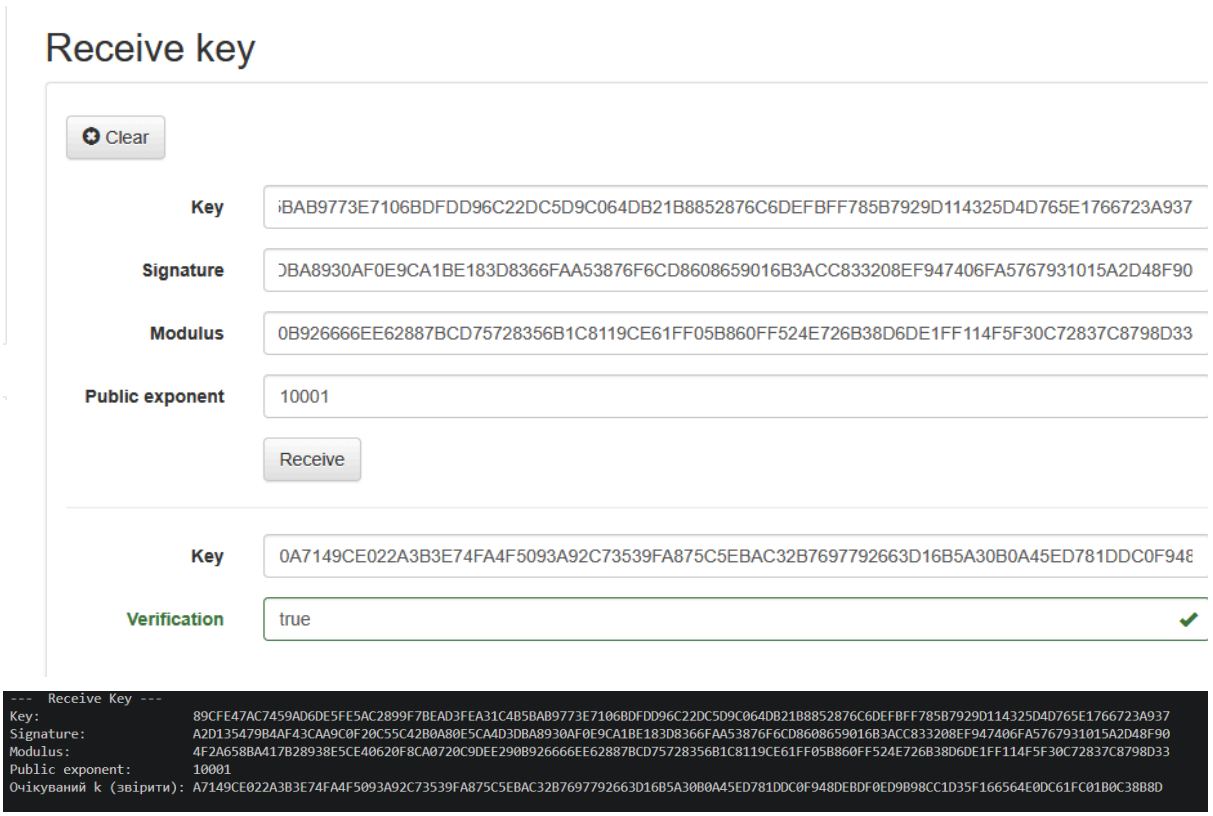



```
--- Decryption ---
Ciphertext : 5DE70E603D5D56E2A3B29F286D3D90A3A86ADC9BEF9E13C2E86DD19392413CE1A31ABB672AB19448A62FA05798D256F2738006FD9785DD0811AE8D6166D5B21F
Очікуваний Message (звірити):  9398C1F7680A43F148CAB58294E385C3441733BD3F0422F52D6DC96E4E1ED39DC47F03FCFF13B50FF86661851ED9C313DF1887713FD27C6AB48F63BA9BD63CBF
```



Підписання працює вірно,

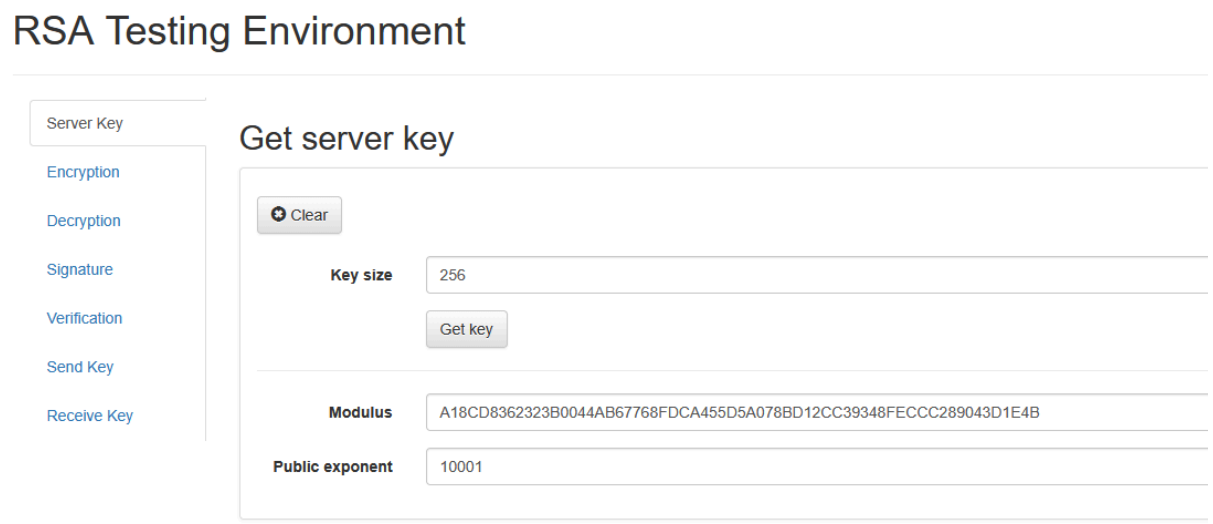
```
---  Verification ---
Message:         19E347AEE671AF31B2B3F06D02D6B6EB9FDB348327E4F6B9D1DA77F28DD0E14927EEBC43B1D4519EDE69BFE00EE77C5A669381A13582D313AE4CDC81BC3E51BE
Signature:       3623A46591B999674DCE9594A0E05A2DE9FF02612EECDA2AEA4A43B377DA953980B92A19F23D166B2737536563EC8F008D8A847A274320893231A75F2DFC1E84
Modulus:         4F2A658BA417B28938E5CE40620F8CA0720C9DEE290B926666EE62887BCD75728356B1C8119CE61FF05B860FF524E726B38D6DE1FF114F5F30C72837C8798D33
Public exponent: 10001
```

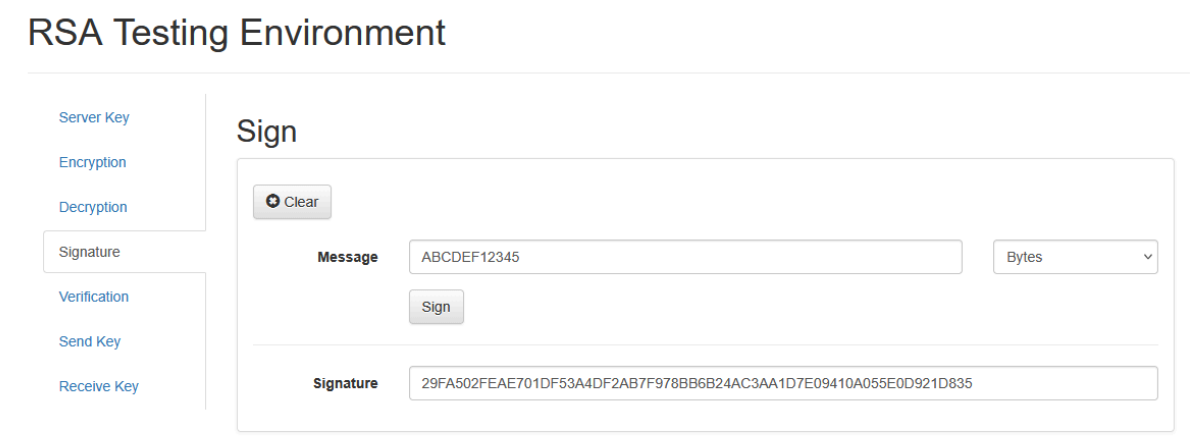


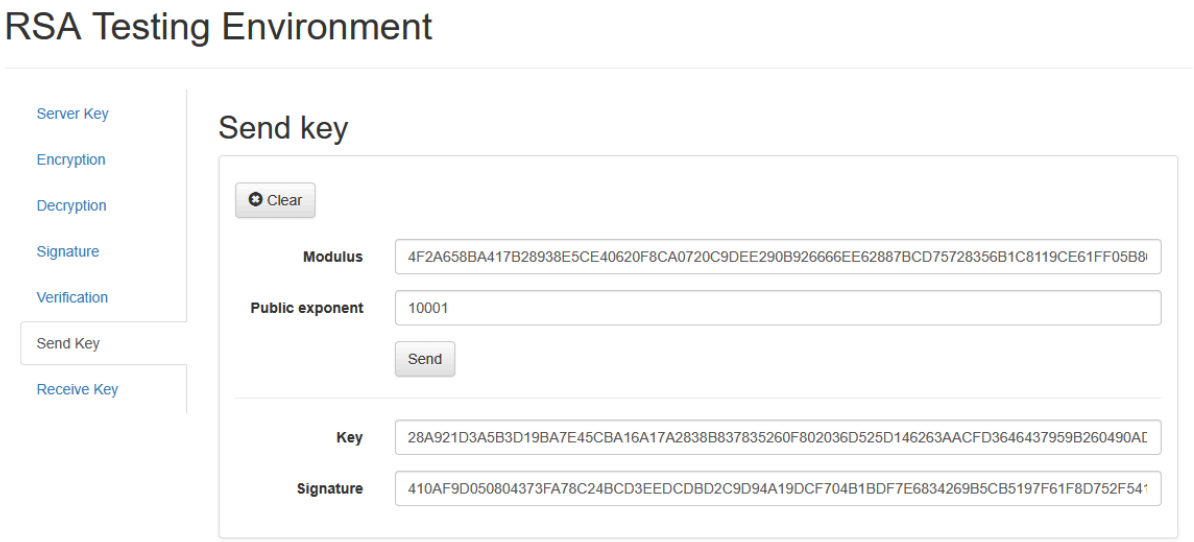

```
---  Receive Key ---
Key:                89CFE47AC7459AD6DE5FE5AC2899F7BEAD3FEA31C4B5BAB9773E7106BDFDD96C22DC5D9C064DB21B8852876C6DEFBFF785B7929D114325D4D765E1766723A937
Signature:          A2D135479B4AF43CAA9C0F20C55C42B0A80E5CA4D3DBA8930AF0E9CA1BE183D8366FAA53876F6CD8608659016B3ACC833208EF947406FA5767931015A2D48F90
Modulus:            4F2A658BA417B28938E5CE40620F8CA0720C9DEE290B926666EE62887BCD75728356B1C8119CE61FF05B860FF524E726B38D6DE1FF114F5F30C72837C8798D33
Public exponent:    10001
Очікуваний k (звірити): A7149CE022A3B3E74FA4F5093A92C73539FA875C5EBAC32B7697792663D16B5A30B0A45ED781DDC0F948DEBDF0ED9B98CC1D35F166564E0DC61FC01B0C38B8D
```

Отриманий ключ також правильний

Тепер реверсивна перевірка








```
--- 5.2. Прийом ключа від сервера (ReceiveKey) ---
Розшифрований ключ k (звірити із сайтом): 1C3D1F335A10CDFD
Справжність відправника підтверджено? -> True
```

Контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?

1. Формування кандидата (старший і молодший біти).
Старший біт встановлюється в 1, щоб число мало рівно 256 біт.
Молодший біт встановлюється в 1, щоб число було непарним, бо парні числа більші за 2 не прості, і перевіряти їх немає сенсу.

2. Пробні ділення на малі прості числа (2, 3, 5, ..., 97).
Це швидкий попередній фільтр. Більшість складених чисел має малий простий дільник, тому вони відсіюються дешевою операцією ділення з остачею, без запуску дорогого

тесту Міллера-Рабіна. Проходження цієї перевірки не доводить простоти, а лише те, що малих дільників немає.

3. Перевірка gcd(x, p) = 1 для кожної основи x (крок 1 тесту Міллера-Рабіна).
Якщо gcd(x, p) > 1, то p ділиться на спільний з x множник, тобто складене. Крім того, умова gcd(x, p) = 1 потрібна для застосування малої теореми Ферма, на якій побудований тест.

4. Перевірка сильної псевдопростоти за основою x (крок 2 тесту Міллера-Рабіна).
Розкладаємо $p - 1 = d \cdot 2^s$ і перевіряємо, що x^d ≡ 1 (mod p) або в ланцюжку x^d, x^(2d), ..., x^(2^(s−1)·d) зустрічається −1. Для простого p це завжди виконується, бо єдині квадратні корені з 1 за простим модулем це ±1. Якщо ж умова порушена, то p точно складене.

5. Повторення для k = 20 різних випадкових основ.
Тест Міллера-Рабіна односторонній: просте число він ніколи не відкине, а складене для однієї основи помилково пропускає з імовірністю не більше 1/4. Повторення k разів зменшує імовірність помилки до $4^{-k} \approx 10^{-12}$.

6. Перевірки на рівні пари чисел.

p ≠ q (для кожної пари): якщо p = q, то $n = p^2$, його легко розкласти, і формула φ(n) = (p − 1)(q − 1) вже не діє.
$pq \le p_1 q_1$: якщо умова порушена, пари міняються місцями. Це потрібно для протоколу розсилання ключів ($n_1 \ge n$).

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?

Умова	**Навіщо потрібна**	**Як перевірено**
p і q прості, p ≠ q	формула φ(n) = (p − 1)(q − 1) та стійкість RSA	тест Міллера-Рабіна (k = 20) у gen_prime; перевірка p ≠ q у distinct_pair
n = p·q	коректний модуль	у check_key: p * q == n
2 ≤ e ≤ φ(n) − 1 і gcd(e, φ(n)) = 1	щоб e мало обернений елемент за модулем φ(n)	у GenerateKeyPair умова перевіряється в циклі вибору e (використовується $e = 2^{16} + 1$); у check_key: gcd(e, phi) == 1
e·d ≡ 1 (mod φ(n))	щоб d був оберненим до e, тобто розшифрування скасовувало шифрування	d обчислюється розширеним алгоритмом Евкліда; у check_key: (e * d) % phi == 1

$(M^e)^d \equiv M \pmod n$	головна властивість RSA: розшифрування повертає вихідне повідомлення	у check_key: випадкове M шифрується й розшифровується, результат порівнюється з M
$n_1 \geq n$ (для протоколу розсилання ключів)	щоб підписане значення вмістилося в модуль адресата	порядок пар забезпечується в generate_primes_for_AB; у виводі n1 >= n: True

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?

Для виконання операцій Encrypt(M, (e, n)) та Verify(M, S, (e, n)) необхідні лише відповідні дані (M або пара M, S) та відкритий ключ (e, n), тоді як для Decrypt(C, (d, n)) і Sign(M, (d, n)) використовуються криптограма C або повідомлення M разом із секретним ключем (d, n). Усі параметри RSA не потрібні для цих операцій, оскільки прості числа p, q та функція Ейлера $\varphi(n)$ застосовуються виключно на етапі генерації ключів для обчислення модуля n і взаємно обернених експонент e та d. Після формування ключів знання p, q і $\varphi(n)$ стає зайвим для обчислень, а принцип асиметричної криптографії вимагає суворого розділення публічного (e, n) та секретного (d, n) ключів між різними учасниками обміну.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника?
У реалізованому протоколі передачі ключа (SendKey / ReceiveKey), побудованому за схемою «спочатку підпис, потім шифрування», властивості безпеки забезпечуються послідовно на двох етапах. На першому етапі відправник A обчислює цифровий підпис своїм секретним ключем (d_A, n_A), що гарантує підтвердження справжності відправника (автентифікацію) та цілісність ключа k, оскільки ніхто інший не володіє секретною експонентою d_A. На другому етапі відправник шифрує як сам ключ k, так і підпис S відкритим ключем отримувача (e_B, n_B), формуючи пакет (k_1, S_1), що забезпечує конфіденційність передачі у відкритому каналі, адже розшифрувати отримані криптограми здатен лише власник секретного ключа d_B.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім?

Перш ніж вважати отриманий ключ k' коректним і справжнім у процедурі ReceiveKey, одержувач B має виконати низку обов'язкових перевірок. Спочатку перевіряється належність вхідних криптограм k_1 та S_1 допустимому діапазону модуля отримувача (

$0 \leq k_1, S_1 < n_B$), після чого вони розшифровуються секретним ключем (d_B, n_B) у пару (k', S'). Далі одержувач контролює коректність меж відновленого ключа ($0 < k' < n_A$) та виконує верифікацію цифрового підпису за допомогою відкритого ключа відправника (e_A, n_A) шляхом перевірки рівності, і лише після успішного підтвердження цієї умови (True) ключ приймається як автентичний.